Università degli Studi di Salerno
Corso di Machine Learning

# Flop Detector

Progetto di Machine Learning

Isabella Dima 0512119719

Anna Chiara Esposito 0512119143

# Contents


**1 Introduzione** **2**

1.1 Scopo del Progetto . . . . . 2

1.2 Scelta del Dataset . . . . . 2

**2 Exploratory Data Analysis (EDA) e Data Cleaning** **3**

2.1 Integrazione dei Dati . . . . . 3

2.2 Parsing e Strutturazione . . . . . 4

2.3 Pulizia del Dataset . . . . . 4

2.3.1 Filtraggio dei Film con Dati Economici Validi . . . . . 4

2.3.2 Creazione della Variabile Target: success . . . . . 4

2.3.3 Feature Engineering Aggiuntivo . . . . . 5

2.4 Ottimizzazione del Dataset . . . . . 5

2.4.1 Rimozione delle colonne non necessarie . . . . . 5

2.4.2 Gestione dei valori nulli . . . . . 5

**3 Statistiche Descrittive Finali** **6**

**4 Feature Engineering** **10**

4.1 Prevenzione del Data Leakage e rimozione colonne non necessarie . 10

4.2 Codifica delle Variabili Categoriche . . . . . 10

4.3 Analisi della Correlazione con il Target . . . . . 11

4.4 Train/Test Split Stratificato . . . . . 12

4.4.1 Visualizzazione del Bilanciamento delle Classi . . . . . 13

4.5 Normalizzazione con StandardScaler . . . . . 13

**5 Addrestramento e Valutazione dei Modelli** **14**

5.1 Selezione degli Algoritmi . . . . . 14

5.2 Addrestramento e Valutazione . . . . . 15
5.2.1 Confronto visivo delle Metriche . . . . . 16
5.2.2 Confusion Matrix dei Modelli Migliori . . . . . 16
5.2.3 Curva ROC . . . . . 16
5.3 Ottimizzazione del Modello migliore con Grid Search . . . . . 17
5.3.1 Valutazione finale del Modello ottimizzato sul Test Set . . . 18
5.4 Feature Importance . . . . . 18
5.5 Conclusioni Finali . . . . . 18

# Chapter 1

# Introduzione

## 1.1 Scopo del Progetto

**Flop Detector** è un progetto di Machine Learning il cui scopo è prevedere di prevedere se un film sarà di successo o meno. Per la buona riuscita del progetto sono state utilizzate tecniche di pulizia dei dati, ottimizzazione e feature engineering. Sono state esplorate diverse opzioni per quanto riguarda i modelli di Machine Learning per determinare il più adatto al nostro problema. I passaggi e le motivazioni saranno descritti nel dettaglio in questo report.

## 1.2 Scelta del Dataset

La nostra scelta è ricaduta su TMDB 5000 Movie Dataset, che è composto da 2 file .csv: tmdb_5000_movies.csv e tmdb_5000_credits.csv.
Il primo comprende 20 colonne con informazioni su budget, generi cinematografici, lingua originale, titolo, indice di popolarità, punteggi e altri attributi, mentre il secondo ha esclusivamente informazioni sulla crew e sul cast. Entrambi i dataset sono dotati di una colonna `id` che permette di unire i due dataset per crearne uno completo di tutte le informazioni necessarie per la riuscita del progetto.

# Chapter 2

# Exploratory Data Analysis (EDA) e Data Cleaning

In questa prima sezione affrontiamo la fase fondamentale di ispezione, pulizia e strutturazione dei dati grezzi.
Gli obiettivi principali di questa fase sono:

(i) **Integrazione dei Dati**: Unione dei dataset `movies` e `credits` tramite chiave univoca (ID del film).

(ii) **Parsing e Strutturazione**: Estrazione delle informazioni chiave (generi, cast, crew, case di produzione) dai formati complessi (JSON/Dizionari) in cui sono memorizzate.

(iii) **Pulizia del Dataset**: Trattamento dei valori nulli tramite mediana e rimozione dei record non validi per l'analisi predittiva (es. film con budget o incassi pari a zero).

(iv) **Ottimizzazione del Dataset**: Rimozione delle variabili testuali non strutturate (come url e descrizioni) per ottenere un dataset pulito e leggero, pronto per le successive fasi di ingegnerizzazione.

## 2.1 Integrazione dei Dati

Sui dataset movies.csv e credits.csv è stata effettuata un'unione sulla base dell'ID del film, ottenendo così un dataset di 4803 righe e 22 colonne.

## 2.2 Parsing e Strutturazione

Nel dataset TMDB sono presenti diverse colonne, tra cui `generi`, `keywords`, `cast` e `crew`, contenenti dati in formato JSON. Per ogni colonna interessata sono state estratte le informazioni principali. In particolare:

- **Generi**: estratti lista completa e genere principale.
- **Keywords**: estratte parole chiave da testo JSON.
- **Case di produzione**: estratte lista completa e crew principale.
- **Paesi di produzione**: estratti paesi da testo JSON.
- **Lingue**: estratte lingue da testo JSON.
- **Cast**: estratti 3 attori protagonisti, tra cui attore principale, e numero totale di attori.
- **Crew**: estratto regista.

## 2.3 Pulizia del Dataset

### 2.3.1 Filtraggio dei Film con Dati Economici Validi

I film i cui valori nelle colonne budget o revenue sono nulli sono stati rimossi, poiché indicano dati sconosciuti o mancanti. Così facendo, il dataset passa da 4803 a 3229 righe.

### 2.3.2 Creazione della Variabile Target: success

Abbiamo definito un film come "di successo" secondo i seguenti criteri:

- **ROI $\geq$ 2.0**: il film ha incassato almeno il doppio del suo budget e ha quindi recuperato i costi di marketing e distribuzione. Questa soglia è comunemente utilizzata nell'industria cinematografica.
- `vote_average` $\geq$ **6.0**: il film ha una valutazione media su TMDB di almeno 6 punti su 10.

Entrambe le condizioni devono essere soddisfatte perché un film sia definito "di successo". Questa doppia condizione cattura sia il successo commerciale che quello di gradimento dal pubblico. Applicati questi criteri, la percentuale di film che è stato categorizzato come "di successo" è stato del 42.6

### 2.3.3 Feature Engineering Aggiuntivo

- Estratta stagione di rilascio dal mese di rilascio del film (`release_season`)
- Estratti numero generi e parole chiave (`n_genres` e `n_keywords`)
- Inserita feature binaria `is_english` che assume valore `true` se il film è in lingua inglese

## 2.4 Ottimizzazione del Dataset

### 2.4.1 Rimozione delle colonne non necessarie

Sono state scartate le colonne JSON originali (ora sostituite da quelle estratte), quelle testuali non strutturate e quelle duplicate. Le colonne rimanenti dopo questa operazione sono 28.

### 2.4.2 Gestione dei valori nulli

I valori null rimasti sono stati gestiti in modo appropriato per ciascuna colonna:

- **Variabili numeriche**: `runtime, popularity, vote_count, vote_average`
  - Imputazione con mediana.
- **Variabili categoriche**: `director, lead_actor, main_company, main_genre, original_language, release_season`
  - Imputazione con "Unknown".

Sono infine state rimosse righe con anno di rilascio sconosciuto.

# Chapter 3

# Statistiche Descrittive Finali

Il dataset pulito conta 3229 righe e 28 colonne.

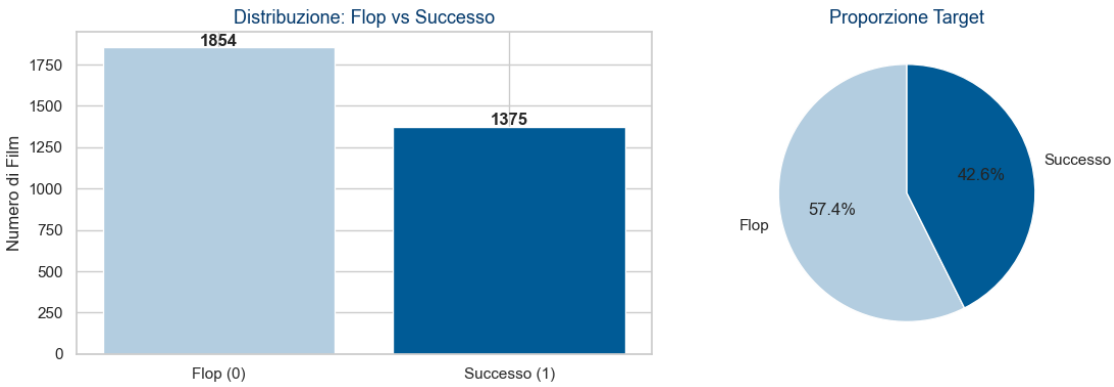


Figure 3.1: Distribuzione della variabile target "success"

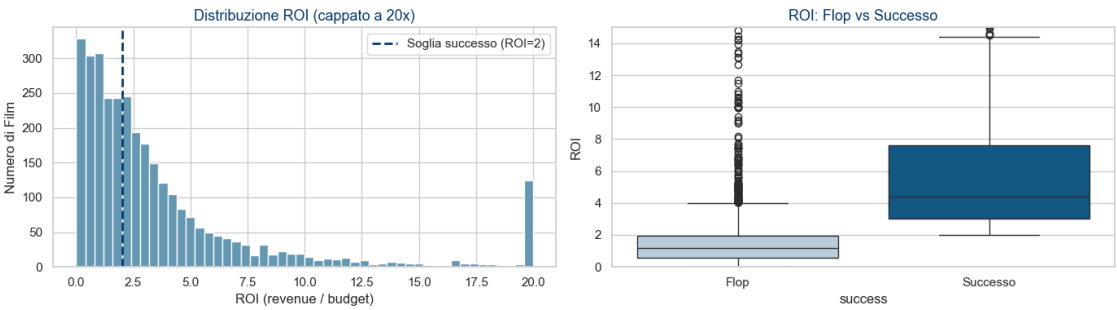


Figure 3.2: Distribuzione del ROI (Revenue / Budget)

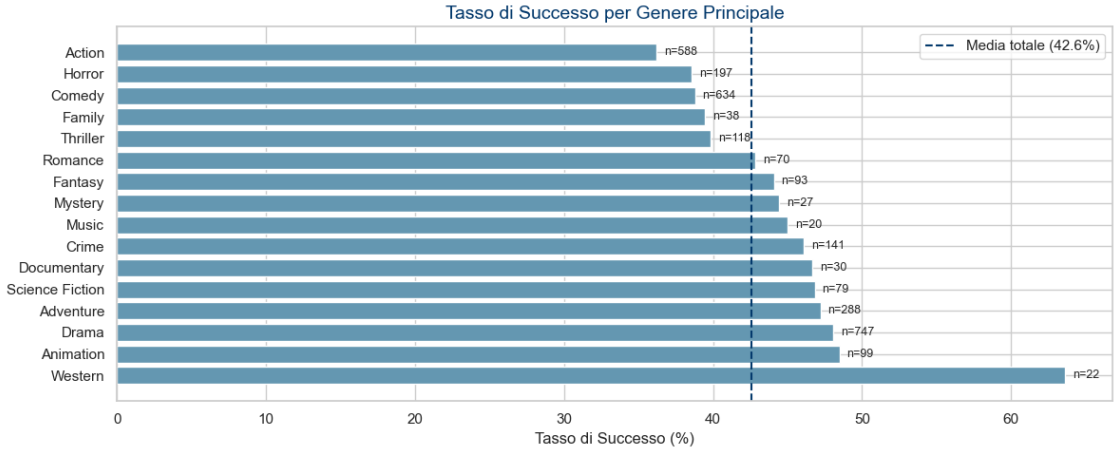


Figure 3.3: Successo per Genere

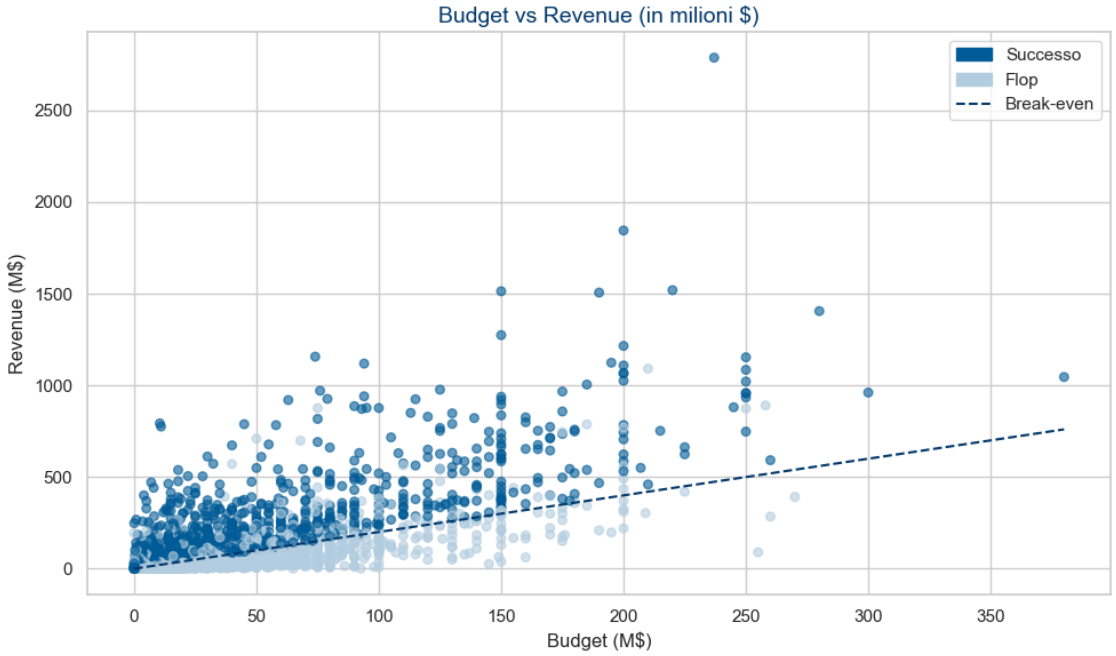


Figure 3.4: Budget vs Revenue

Scatterplot per visualizzare la relazione tra budget investito e ricavi ottenuti, colorato per variabile target.

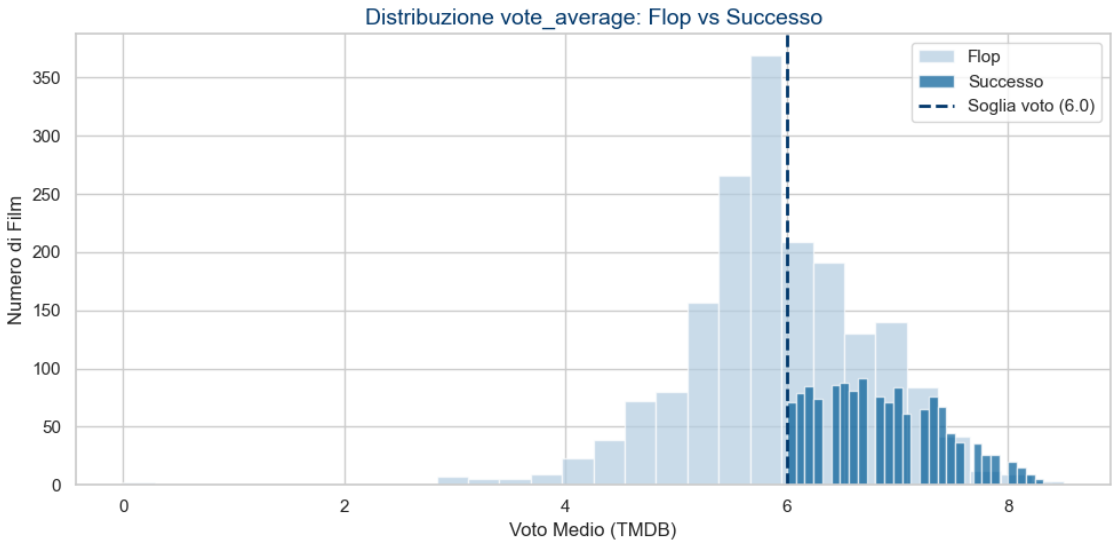


Figure 3.5: Distribuzione di vote˙average per Successo / Flop

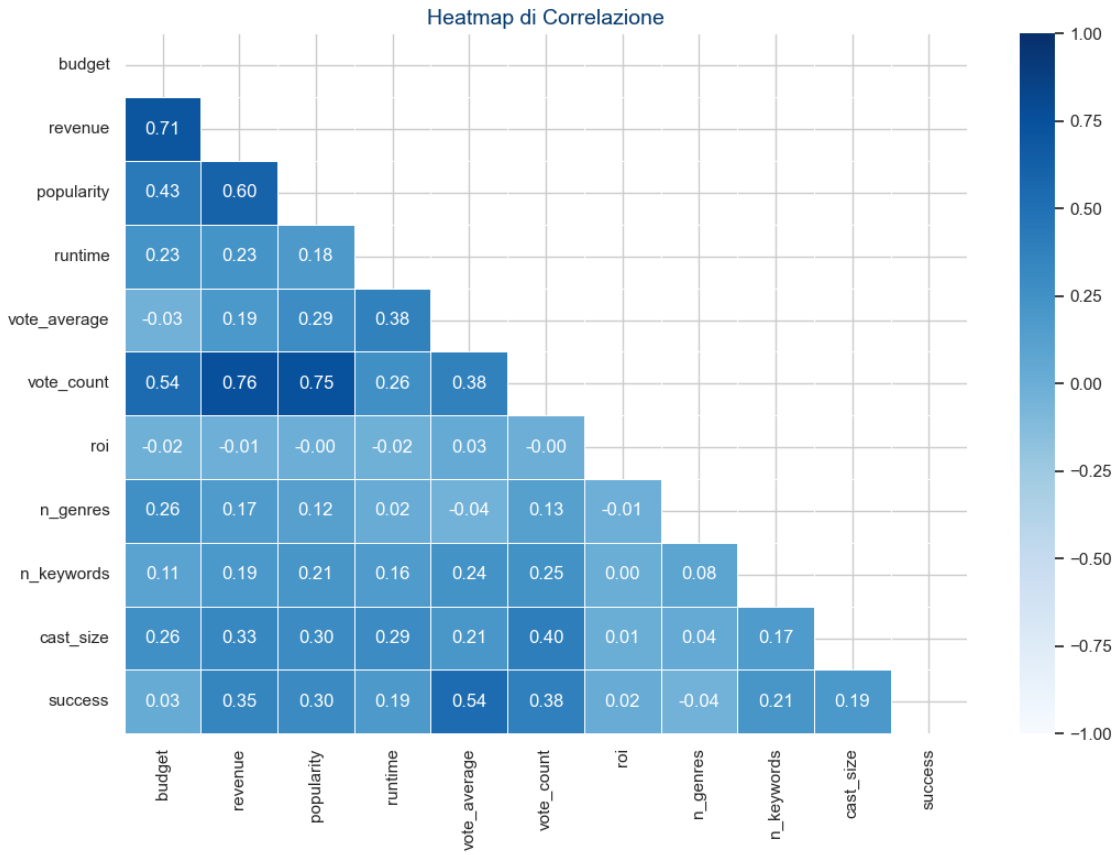


Figure 3.6: Heatmap di Correlazione tra Feature numeriche

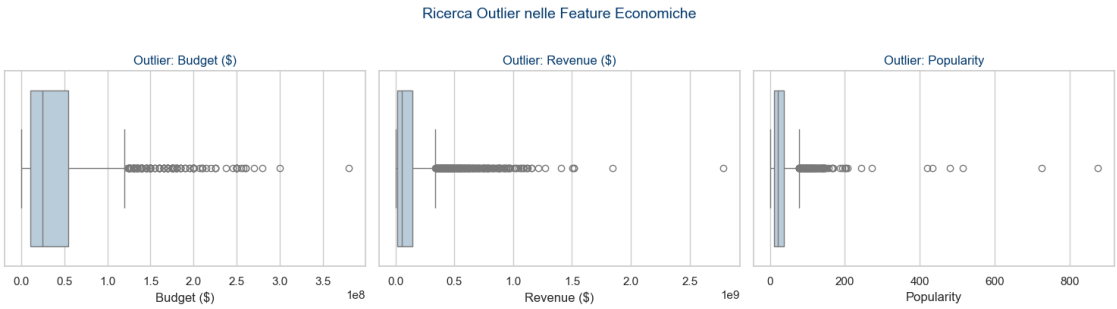


Figure 3.7: Boxplot degli Outlier sulle Feature numeriche chiave

Il boxplot degli incassi (revenue) mostra una distribuzione fortemente asimmetrica a destra (right-skewed): la mediana si attesta intorno ai 50-60 milioni, mentre i grandi blockbuster (Avengers, Avatar) generano outlier estremi nell'ordine dei miliardi. Questo giustifica l'utilizzo della mediana (invece della media) per l'imputazione.

# Chapter 4

# Feature Engineering

## 4.1 Prevenzione del Data Leakage e rimozione colonne non necessarie

La variabile target `success` è stata costruita a partire da ROI (calcolato da `revenue`) e da `vote_average`. Queste colonne, essendo dati calcolati successivamente all'uscita del film, non saranno utilizzate come feature di input, in quanto il modello potrebbe dedurre la risposta corretta anche solo da queste colonne.
Saranno invece mantenute le colonne `budget`, che indica l'investimento a priori, e `popularity`, un indice TMDB che è legato all'hype pre-uscita.
Inoltre, sono state rimosse alcune colonne non necessarie, quali `id`, `title`, `original_language`, `genres_list`, `keywords_list`, `companies_list`, `countries_list`, `languages_list` e `top3_cast`.

## 4.2 Codifica delle Variabili Categoriche

Per tramutare le variabili categoriche in input numerici adotteremo strategie diverse in base alla cardinalità:

- `main_genre` e `release_season`: cardinalità bassa ($\leq 20$ valori) $\rightarrow$ **One-Hot Encoding**
- `director`, `lead_actor`, `main_company`: cardinalità alta (centinaia di valori) $\rightarrow$ **Target Encoding**

Il **One-Hot Encoding** crea una nuova colonna per ogni categoria e assegna 1 se la categoria è presente, 0 altrimenti. Questo è ideale per categorie con cardinalità ridotta, così da non introdurre troppe colonne aggiuntive.
Il **Target Encoding**, invece, sostituisce ogni categoria con la percentuale storica di successo di quella categoria. Cattura il "peso" di un regista o attore senza far esplodere le dimensioni del dataset.
Queste operazioni ci hanno portato a un totale di 37 colonne.

## 4.3 Analisi della Correlazione con il Target

Prima dello split abbiamo analizzato visivamente quali feature hanno la maggiore correlazione lineare con la variabile target. Questo ci ha consentito di avere un'indicazione delle feature più predittive.

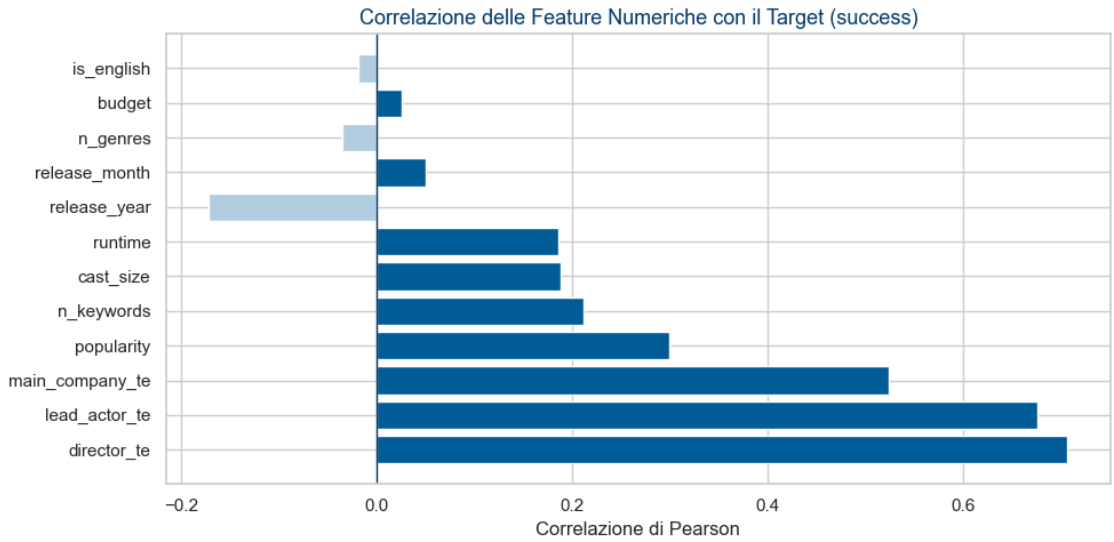


Figure 4.1: Correlazione delle Feature Numeriche con il Target (success)

| Colonna | Correlazione |
|---|---|
| director_te | 0.707291 |
| lead_actor_te | 0.675951 |
| main_company_te | 0.524124 |
| popularity | 0.299585 |
| n_keywords | 0.211218 |
| cast_size | 0.188742 |
| runtime | 0.186349 |
| release_year | -0.172107 |
| release_month | 0.050570 |
| n_genres | -0.035063 |
| budget | 0.025345 |
| is_english | -0.019074 |

Table 4.1: Correlazione ordinate per valore assoluto

## 4.4 Train/Test Split Stratificato

Separiamo le feature (`X`) dal target (`y`) ed eseguiamo uno split **80% training / 20% test**. Usiamo `stratify=y` per garantire che entrambi i set mantengano la stessa proporzione di film di successo rispetto al dataset originale, dato che le classi non sono perfettamente bilanciate.

### 4.4.1 Visualizzazione del Bilanciamento delle Classi

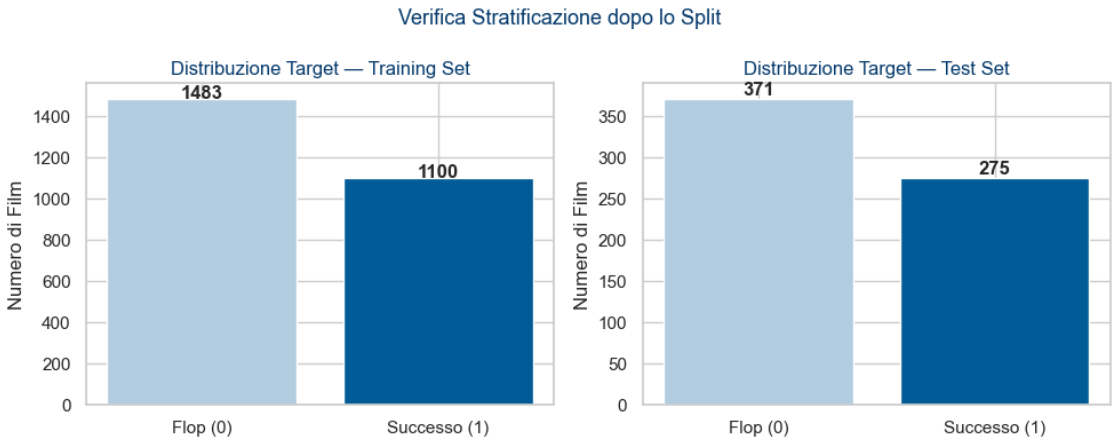


Figure 4.2: Verifica Stratificazione dopo lo Split

## 4.5 Normalizzazione con StandardScaler

Alcuni modelli (Logistic Regression, SVM) sono sensibili alla scala delle feature: variabili con range molto diversi (budget in milioni vs runtime in minuti) possono distorcere l'apprendimento.

Abbiamo applicato dunque `StandardScaler` che porta ogni feature a media 0 e deviazione standard 1.

# Chapter 5

# Addrestramento e Valutazione dei Modelli

Questa sezione si occuperà della fase di **Modeling** e **Evaluation**.
L'obiettivo è addestrare diversi algoritmi di Machine Learning per risolvere un problema di**classificazione binaria**: prevedere se un film avrà successo ('1') o sarà un flop ('0') basandosi sulle sue caratteristiche **a priori** (budget, genere, regista, stagione di uscita, ecc.).
Confronteremo le prestazioni dei modelli attraverso metriche robuste e analizzeremo quali feature risultano più predittive.

## 5.1 Selezione degli Algoritmi

Abbiamo testato diversi per capire quale riesce a prevedere meglio il successo di un film:

(i) **Logistic Regression**: Il modello più semplice e lineare, che usiamo come punto di partenza (**baseline**).

(ii) **Decision Tree (Albero Decisionale)**: Crea una serie di domande a cascata (es. "Il budget è ¿ 50M?"). Presenta un elevato rischio di imparare i dati "a memoria" (*overfitting*).

(iii) **Random Forest**: Unisce le forze di decine di alberi decisionali, risultando molto più stabile e preciso.

(iv) **SVM (Support Vector Machine)**: Traccia un confine matematico netto per separare i Flop dai Successi.

(v) **XGBoost (Extreme Gradient Boosting)**: L'algoritmo più avanzato della lista, infatti impara progressivamente dai propri errori. Costruisce un *ensemble* di decision trees in maniera sequenziale, dove ogni nuovo albero corregge gli errori dei precedenti.

Per compensare il leggero sbilanciamento delle classi, abbiamo utilizzato `class_weight='balanced'` dove disponibile.

## 5.2 Addrestramento e Valutazione

Per ogni modello sono stati calcolati:

- **Accuracy**: percentiale di previsioni corrette.
- **Precision**: dei film previsti come successo, quanti li sono davvero?
- **Recall**: dei film veramente di successo, quanti ne riusciamo a identificare?
- **F1-Score**: media armonica di precision e recall, utile con classi sbilanciate.
- **ROC-AUC**: area sotto la curva ROC, misura la capacità discriminativa del modello.
- **CV F1 (5-fold)**: F1 media su 5 fold di cross-validation, più robusta del semplice test set.

| | **Accuracy** | **Precision** | **Recall** | **F1-Score** | **ROC-AUC** | **CV F1 (5-fold)** |
|---|---|---|---|---|---|---|
| **XGBoost** | 0.9241 | 0.8924 | 0.9345 | 0.9130 | 0.9791 | 0.9006 |
| **Random Forest** | 0.9195 | 0.8832 | 0.9345 | 0.9081 | 0.9725 | 0.9015 |
| **SVM** | 0.9149 | 0.8642 | 0.9491 | 0.9047 | 0.9715 | 0.8824 |
| **Logistic Regression** | 0.9118 | 0.8682 | 0.9345 | 0.9002 | 0.9730 | 0.8884 |
| **Decision Tree** | 0.8793 | 0.8456 | 0.8764 | 0.8607 | 0.8789 | 0.8472 |

### 5.2.1 Confronto visivo delle Metriche

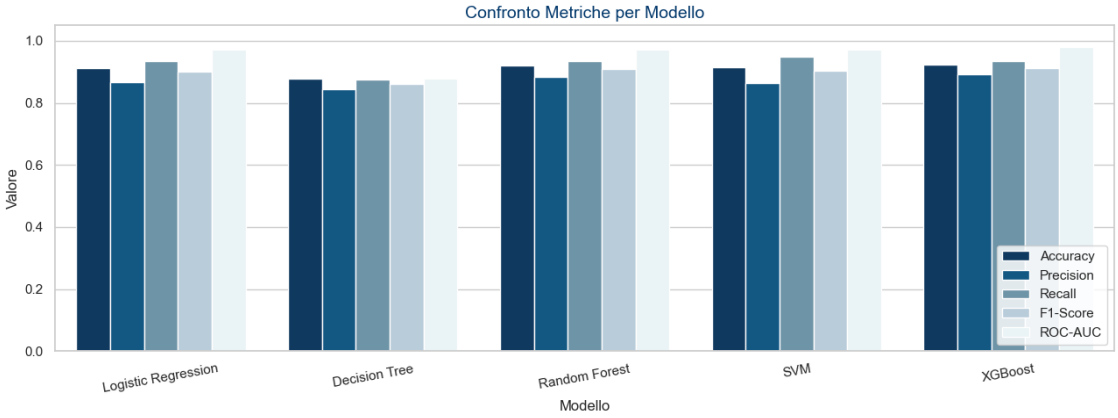


Figure 5.1: Confronto Metriche per Modello

### 5.2.2 Confusion Matrix dei Modelli Migliori

La confusion matrix mostra nel dettaglio dove ogni modello sbaglia:

- **Falsi Negativi (FN)**: film di successo classificati come flop (perdiamo opportunità).
- **Falsi Positivi (FP)**: film flop classificati come successo (sovrastimiamo).

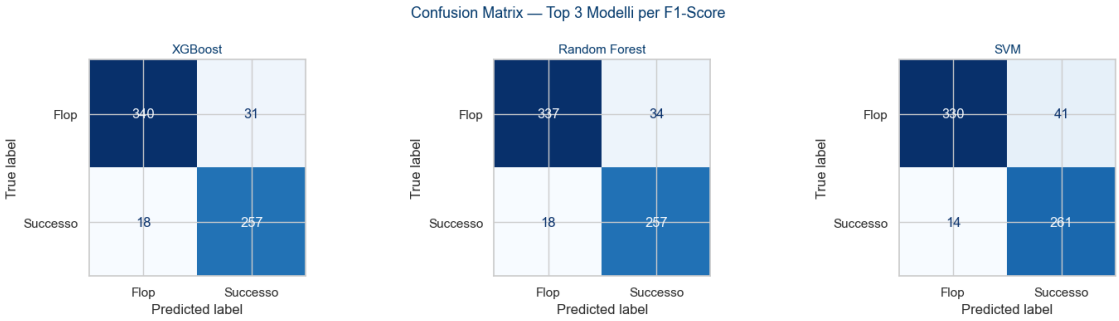


Figure 5.2: Confusion Matrix per modelli XGBoost, Random Forest e SVM

### 5.2.3 Curva ROC

La curva ROC (Receiver Operating Characteristic) mostra il trade-off tra **True Positive Rate** (recall) e **False Positive Rate** al variare della soglia di classificazione. Un modello perfetto avrebbe AUC = 1.0; uno casuale avrebbe AUC = 0.5.

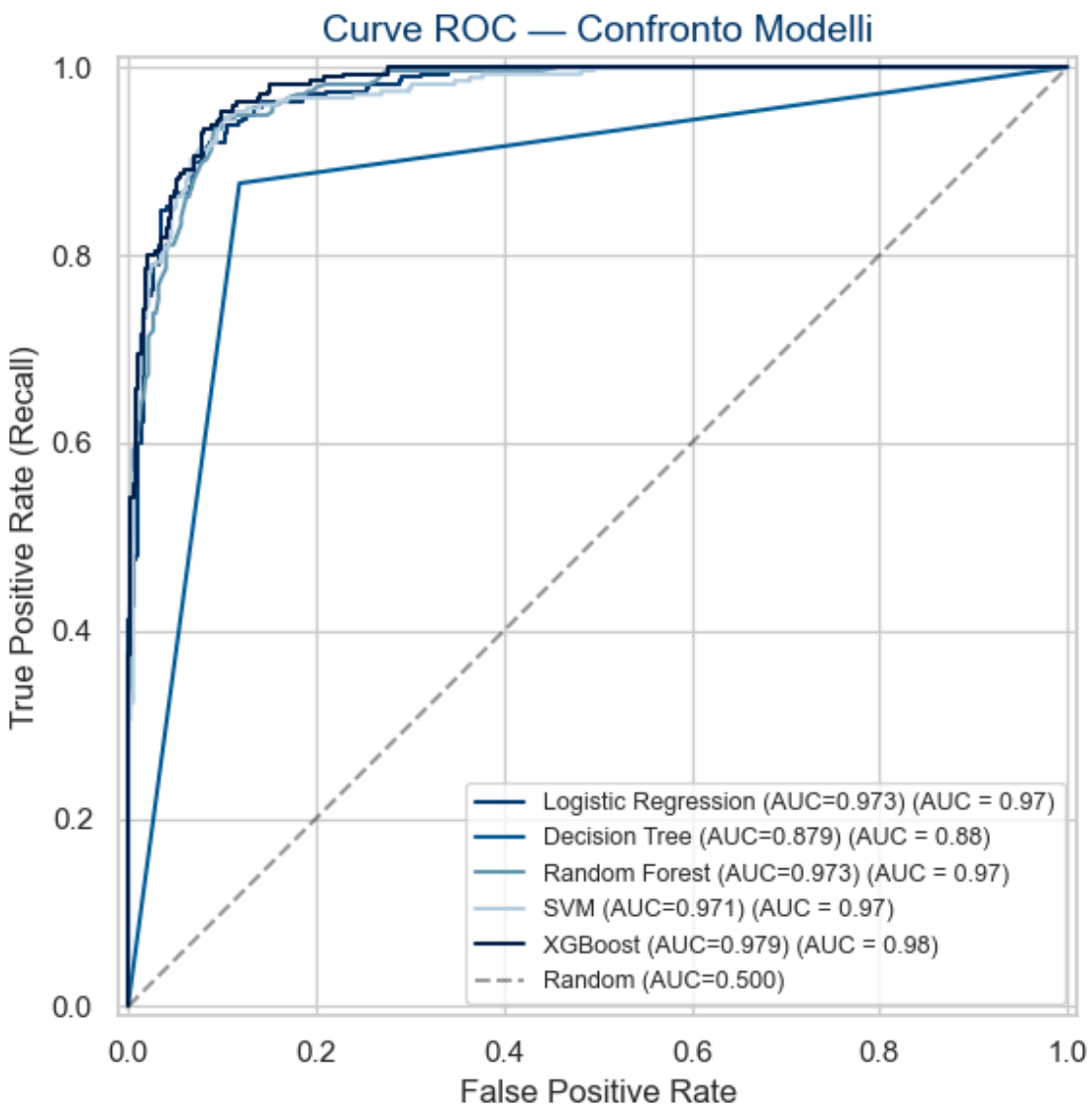


Figure 5.3: Curva ROC

## 5.3 Ottimizzazione del Modello migliore con Grid Search

Il modello migliore, in base al F1-Score, si è rivelato **XGBoost**. Per ottimizzarlo ulteriormente abbiamo effettuato una **Grid Search** con **Cross-Validation a 5 fold** per trovare i migliori iperparametri.

La cross-validation divide il training set in 5 parti: addestra su 4 e valida sulla quinta, ripetendo il processo 5 volte. Questo produce una stima più robusta rispetto al semplice test set.

### 5.3.1 Valutazione finale del Modello ottimizzato sul Test Set

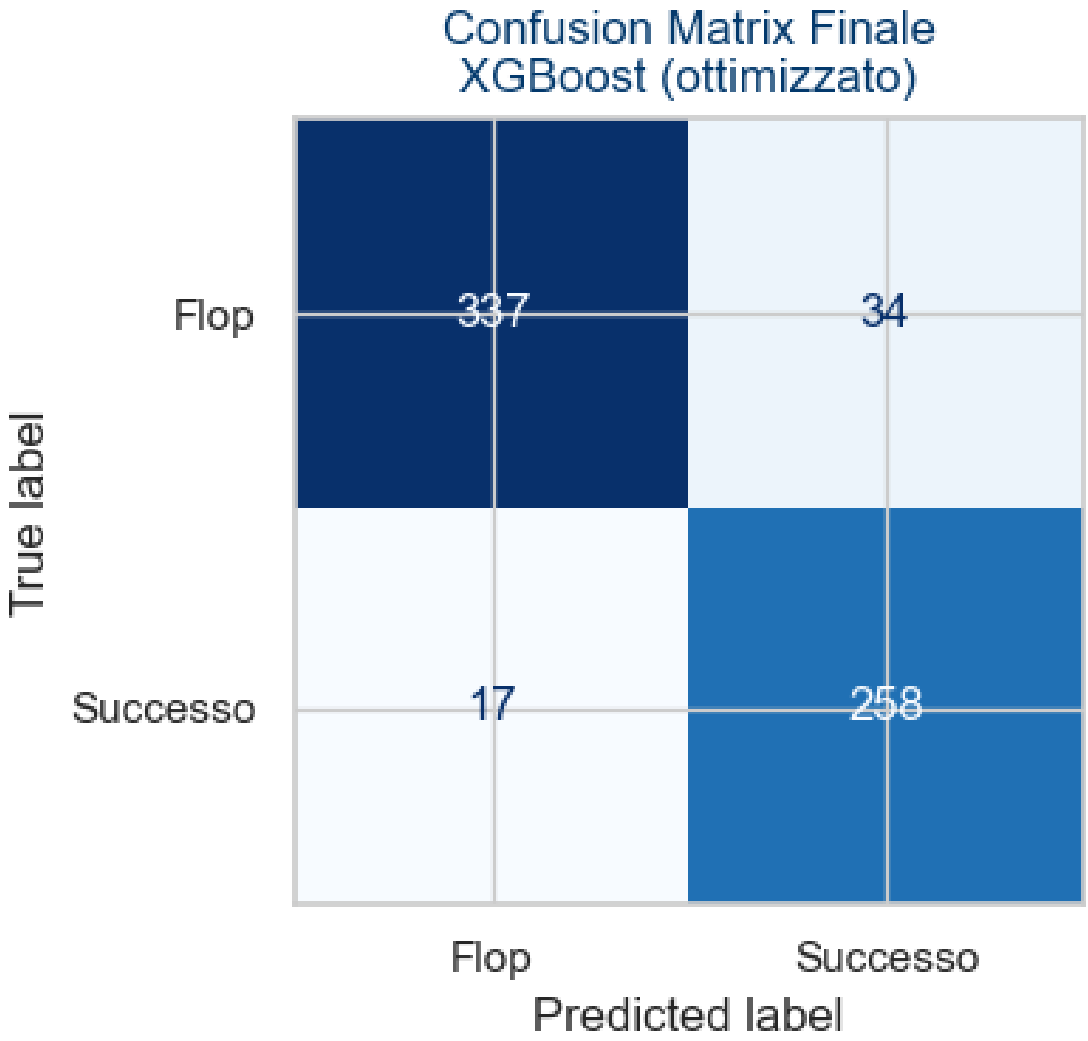


Figure 5.4: Confusion Matrix Finale XGBoost (ottimizzato)

## 5.4 Feature Importance

Analizziamo quali feature hanno avuto il maggior peso nella decisione del modello. Questo ci permette di capire quali caratteristiche di un film sono davvero predittive del suo successo.

## 5.5 Conclusioni Finali

Il progetto ha testato la capacità predittiva del Machine Learning nel mercato cinematografico usando esclusivamente caratteristiche *a priori* — informazioni

| Feature | Importance |
|---|---|
| director_te | 0.358563 |
| lead_actor_te | 0.184219 |
| main_company_te | 0.097436 |
| popularity | 0.083791 |
| budget | 0.054144 |
| release_year | 0.035199 |
| cast_size | 0.029508 |
| n_genres | 0.026443 |
| n_keywords | 0.026295 |
| release_month | 0.022966 |
| runtime | 0.017126 |
| main_genre_Action | 0.016344 |
| main_genre_Horror | 0.013586 |
| release_season_Fall | 0.013147 |
| main_genre_Adventure | 0.010875 |

disponibili prima che il film venga rilasciato.

Dall'analisi delle feature sono stati evidenziati questi aspetti:

- **Budget e Popularity** dominano la capacità predittiva: i film con maggiori investimenti e più hype pre-uscita tendono significativamente al successo.
- **Il Target Encoding di regista e attore protagonista** risulta informativo: la storia di successo di chi c'è dietro e davanti alla macchina da presa è predittiva.
- **Il genere ha un impatto limitato**: rispetto alle variabili economiche: il successo trascende la categoria tematica del film.
- **La stagione di uscita** (estate e dicembre/inverno) mostra un lieve vantaggio, coerente con le strategie di uscita dei blockbuster.

Il modello, tuttavia, presenta alcuni limiti. I fattori più predittivi del successo (qualità della sceneggiatura, marketing, word-of-mouth, concorrenza al botteghino) sono difficilmente quantificabili e non presenti nel dataset. Inoltre, il modello

cattura strutture statistiche aggregate, e non la qualità intrinseca di un'opera.